

A View of Cloud Computing

Cloud computing refers to both applications delivered as services and the hardware and systems software in the data centers that provide these services. The paper focuses on SaaS providers, which are cloud users, and cloud providers. In the context of the paper, as long as resource allocation can track expected load variances, it is considered cloud computing, which excludes private clouds.

The author discusses three classes of cloud computing. EC2 is on one extreme of the spectrum, which provides hardware virtualization and users can control the entire software stack, which makes it inherently difficult for Amazon to provide autoscaling and failover. Other extreme is Google's AppEngine which provides clean separation between stateless computation tier and stateful storage tier. Microsoft azure lies in between as it provides flexibility but constraints user's choice of storage model and app structure.

The paper offers three usecases in favor of utility computing over conventional hosting. Firstly, datacenters usually have to provision for peak load, which means they are underutilized for peak load. Hence using cloud makes more economic sense due to its flexible pricing. Secondly, for startups and seasonal businesses, demand is usually unknown in advance which makes it difficult to estimate the hardware needed. Third argument is cost associativity. For large workloads, running 1000 EC2 machines costs the same as running 1 machine for 1000 hours. The author argues that even if Amazon's pay-as-you-go model was more expensive, the cost is outweighed by the transfer of risk, especially of under or over provisioning. Fine grained control over adding or removing servers in minutes allows matching resources to workload more efficiently.

Lastly, the author discusses the obstacles to cloud computing. Users are wary of cloud computing because 1. what if the provider goes out of business, 2. storage APIs are non standardized and proprietary, making it difficult to switch providers, 3. trusting someone else with your data, especially in cases of HIPAA or other compliance issues, 4. data transfer bottlenecks; it is faster to ship data on hard disks, 5. performance unpredictability, 6. lack of storage scalability, 7. risks of bugs in large scale distributed systems which are difficult to debug and resolve, 8. idle costs - pay as you go applies to storage and networking, not to computation, 9. reputation free fate sharing - one bad actor can cause blacklisting of EC2 IPs, affecting all other users and 10. Software licensing agreements, which restrict the computers on which software can run.

One critique here is that the paper was written in 2010 and the cloud computing landscape has evolved immensely since then. For instance, while Amazon's EC2 pricing model is largely unchanged, it is now a small part of the services that AWS offers. Elastic Beanstalk and Cloudformation now provide template based solutions to manage resources, allowing load balancing, autoscaling, faster storage options and cheaper compute options (like arm based CPUs instead of x86 processors). Another thing that I found interesting is the HIPAA compliance mentioned in obstacles to cloud computing. With acts like GDPR in place, it would be interesting to study how they affect the usage of cloud services and user's trust.

A Guided Tour of Data-Center Networking

The paper provides an inside view of a data center, focusing particularly on the networking aspect. A modern day data center is home to thousands of hosts, each containing one or more processors, memory, network interface and local high speed IO. Compute resources are packaged into racks and allocated as cluster consisting of thousands of high speed network connected hosts. The network, despite being only 10-15% of the cost, acts a backbone of the data center.

Tasks are distributed amongst the hosts and are executed in parallel, with the maximum response time of any subtask dictating the overall response time. This means that communication overhead between the hosts can limit application performance. Web applications are usually built with shared resources in mind while high performance computing applications require dedicated systems. In a data center, network over-provisioning can reduce the likelihood of congestion, however, it is extremely expensive. Therefore, QoS policies are implemented to segregate traffic into distinct classes, usually at the switch and NIC level. The traffic is characterized according to flows and is usually classified into "elephant" and "mice" flows. Elephant flows are smaller in number but contain large number of packets and can have a huge performance impact. The paper also states that the traffic between clusters usually consists of much larger payloads. Intracluster traffic is usually more fine grained with bursty behavior.

The paper details the data center network architecture, where the topology is commonly represented as graphs with switches and hosts represented by vertices and links represented by edges. The architecture also consists of different types of unique addresses at different layers, like IP at layer 3 and ethernet at layer 2, and various routing methods. Protocols are in place to avoid transient routing loops. Source routing is less flexible and has an overhead of discovering all paths ion the network. Distributed lookups provide more flexible routing.

The paper also discusses the etiquette for sharing network resources like physical links and buffer space. Flow control dictates how input buffer at each switch or nic are managed. Congestion control is in place to avoid saturation. User-kernel context switching in TCP/IP/Ethernet creates a critical latency bottleneck. To fix this, OS bypass or zero copy message passing protocols can be employed. The former deposits messages directly into a user-application buffer while the latter avoids memory copy from user to kernel space and vice versa. Cluster ecosystem requires flexibility as top priority to ensure scalability. Each successive layer in network stack enables abstraction over the other providing platform independent access to both data and system measurement. Fault recovery mechanisms are also incorporated to enable reliability, adding further complexity to the architecture.

I found this paper to be very insightful, as it provides an overview of the complex engineering that powers the internet. The author details different aspects of the datacenter, including the ones that would otherwise not cross someones mind like the significance of racks and host placement and all the protocols and tools in place to serve modern web needs.